

Hiring Assignment: Fine-Tune & Self-Host a Small Language Model

Objective

Fine-tune an open-source small language model (SLM) on the dataset below so that it answers customer-support questions better than the base model does, and self-host it behind an API.

Every meaningful choice is yours - model, size, fine-tuning method, data handling, evaluation design, serving stack. We want to see how you decide, not how you follow instructions.

Use Case

A customer messages a business with a support request (account access, orders, refunds, payments, invoices, subscriptions, etc.) and the model responds as the support assistant.

The fine-tuned model should:

- Understand the customer's request
 - Respond accurately and in a helpful support tone
 - Do this measurably better than the base model it started from
-

Dataset

Bitext Gen AI Chatbot Customer Support Dataset (Kaggle)

kaggle.com/datasets/bitext/bitext-gen-ai-chatbot-customer-support-dataset

- 26,872 instruction/response pairs (~3.57M tokens)
- Fields: flags, instruction, category, intent, response
- 27 customer-support intents across 10 categories (Account, Order, Refund, Payment, Delivery, Invoice, Subscription, and more)

How you use it is up to you - all of it or a subset, which columns matter, and how you split it (the dataset is paraphrase-heavy; be careful about leakage). You may bring in additional data if you want; one option is **Customer Support on Twitter**

(kaggle.com/datasets/thoughtvector/customer-support-on-twitter). Entirely optional.

Requirements

Setup

- An open-source SLM with a license that permits commercial self-hosting - your pick, justify it
 - A parameter-efficient or full fine-tuning method - your pick, justify it
 - Free compute (Kaggle / Colab) is sufficient; do not spend money on this
 - Self-hosted inference: the final model served behind an HTTP API (vLLM, SGLang, TGI, llama.cpp/Ollama, or your choice)
-

Core Work

1. Data Preparation

- Explore the dataset and prepare it for your model's chat/instruction template
 - Create leakage-free train / validation / held-out test splits
-

2. Fine-Tuning

- Train the model; log and inspect training/validation curves
 - Document your hyperparameter choices and what you tried
-

3. Evaluation (most important)

Convince a skeptical engineer the improvement is real:

- Define what "better" means for this task - metrics, methods, and judge (automatic, model-based, human, or a mix) are your choice
 - Measure on data the model never trained on
 - Compare base model vs. fine-tuned model, including failure cases
-

4. Serving (Self-Hosting)

- Serve the fine-tuned model behind an HTTP API
 - Report basic latency/throughput on your hardware and how you measured it
-

Deliverables

1. Code & Implementation

- Git repository with the full pipeline: data prep, training, evaluation, serving

- Fine-tuned weights or adapter (Hugging Face Hub link or in the repo)
 - Steps to install and run everything locally
-

2. Loom Demo (2-5 Minutes)

Record a Loom (or equivalent screen recording) showing:

- Your served endpoint answering live customer-support queries
 - A side-by-side: base model vs. your fine-tuned model on the same queries
 - A walkthrough of your evaluation results
 - At least one failure case and what you learned from it
-

3. Documentation (README)

Include:

- Model and method choices, and why
 - Data handling and split strategy
 - Evaluation design and results
 - The exact prompt template you trained with, and how to load and run inference on your model
 - What's real vs. cut for time, and trade-offs you'd revisit for production
-

Evaluation Criteria

- **Evaluation Rigor:** honest, leakage-free base-vs-tuned comparison with defensible metrics
- **ML Judgment:** model, method, and hyperparameter choices are reasoned, not copied
- **Systems Thinking:** working self-hosted endpoint with credible performance numbers
- **Reproducibility:** we can re-run your pipeline from the README
- **Code Quality:** modular and maintainable

Note: we maintain our own internal, held-out evaluation set of customer-support queries. We will run your submitted model against it, and its performance there - alongside your reported results - will form the basis of our assessment. Make sure your model is easy for us to load and run with your documented prompt template. A model that only shines on its own test split will not hold up.

Submission

- GitHub repo

- Loom demo video
- README
- Model weights / adapter link

Suggested effort: 3-5 days part-time; submit within 7 calendar days. AI assistants are allowed - you must be able to defend everything you submit in the follow-up interview. Questions are welcome.